Optimization and Experimental Evaluation of the SOX Fair Exchange Implementation

Ryad Mohamed Aouak

School of Computer and Communication Sciences

Semester Project

Spring 2026

Responsible
Prof. Serge Vaudenay
EPFL / LASEC

Supervisor
Prof. Serge Vaudenay
EPFL / LASEC

Contents

Chapter 1

Introduction

1.1 Context and motivation

Fair exchange is the problem of allowing two parties to exchange valuable assets in such a way that no honest participant is left at a disadvantage. In the setting considered in this project, a vendor V owns a digital item x, a buyer B wants to buy it for a price y, and both parties know in advance a public description predicate desc and an expected digest d. The intended outcome is simple to state: either B obtains an item x' such that $\mathsf{desc}(x') = d$ and V receives the payment, or the exchange is cancelled without giving either side an unfair advantage.

This problem is classical in cryptography, but it remains difficult to deploy in practice. Without an external trusted party, perfect two-party fair exchange is impossible in the general case. Smart contracts provide a natural alternative: the blockchain can act as a public, deterministic, and economically enforceable adjudicator. However, using a blockchain introduces another difficulty. Every on-chain action costs gas, and these costs can make the exchange unattractive, especially when the exchanged item is small, when one participant has no cryptocurrency available, or when a dispute forces expensive verification on-chain.

The Sponsored Fair Exchange protocol, denoted SOX in this report, addresses this tension by combining three ideas. First, it follows an optimistic design: when both parties behave honestly, the exchange should finish with very little on-chain computation. Second, it keeps a dispute mechanism able to resolve disagreements by verifying only a logarithmic number of positions in a computation trace. Third, it introduces sponsors, whose role is to cover gas costs under predefined incentives. The goal is not only fairness, but also economic accessibility: the vendor and the buyer should not need to manage gas in the common case.

The starting point of this semester project was an existing implementation of SOX, documented in the previous implementation report [2] and based on

the protocol paper [1]. That implementation already represented a substantial proof of concept. It contained a Rust/WebAssembly off-chain pipeline, Solidity contracts for the optimistic and dispute phases, an Electron/Next.js interface, and an ERC-4337-inspired transaction sponsorship path. It also introduced a compact block-based circuit representation for large files, with 64-byte gates and Merkle commitments over the circuit, ciphertext, and intermediate values.

The purpose of the present project is different from reimplementing SOX from scratch. The objective is to take this initial system seriously as a research prototype, analyze where its implementation diverges from the most gas-efficient protocol variants, and modify it so that the main variants suggested by the protocol analysis can be executed and measured. In other words, the project is both an implementation project and an experimental methodology project: the code must work, but the measurements must also correspond to well-defined protocol scopes.

1.2 Terminology used throughout the report

A recurring source of confusion in this project is that several implementations coexist in the repository and they do not measure the same thing. To avoid ambiguity, this report uses the following terminology consistently.

This distinction is essential for interpreting the experimental results. A number such as “optimistic cost”, “deployment cost”, or “dispute cost” is meaningful only after saying which implementation is measured and which protocol steps are included. One of the outcomes of this project is therefore a stricter accounting discipline: costs are reported either as one-time infrastructure deployments, per-exchange optimistic costs, dispute initialization costs, or per-round dispute execution costs.

1.3 Project goals

The project was initially motivated by the need to understand the existing SOX implementation and to make it executable on a complete local stack. After this first phase, the scope was refined around gas reduction and protocol variants, following discussions with the supervisor. The final goals can be summarized as follows.

1. Reconstruct and validate the execution path of the inherited implementation, from off-chain precontract generation to optimistic completion and dispute execution.

2. Identify which sponsoring variants actually affect gas costs. In particular, the project studies no S-deposit, $S = B$, $S = V$, $S_B = B$, and $S_V = V$,

Term	Meaning in this report
Initial implementation	The implementation inherited from the previous semester project. It is the baseline used to understand the existing architecture and to reproduce the original costs.
Monolithic implementation	The intermediate implementation in which several variants are supported by the same large contracts. This version is useful for validation but expensive because variant selection logic remains on-chain.
Specialized final implementation	The optimized implementation produced in this project, where the main protocol modes are split into smaller dedicated contracts or clone-based variants. The goal is to move mode selection off-chain and avoid paying gas for unused branches.
Hardcoded SHA-256 mode	A specialized dispute mode for the case $\mathsf{desc} = \mathsf{SHA256}$, where the circuit is determined by the input length and no generic circuit commitment proof π_1 is required in Step 8.
simSOX / no S-deposit	A simplified optimistic mode in which there is no initial sponsor deposit by S. Each participant pays gas in the optimistic part, removing the need for sponsor reimbursement logic in that part of the contract.

Table 1.1: Implementation terminology used in the report.

while separating variants determined at precontract time from variants determined only once a dispute starts.

3. Implement the main variants in Solidity without changing the intended protocol semantics. The implementation must avoid a "Swiss-army-knife" contract where all modes are present in one large bytecode and all users pay for unused logic.

4. Implement and measure a hardcoded SHA256 dispute mode. For this mode, the circuit is fixed by $|x|$, so the smart contract can reconstruct the relevant gate in Step 8 instead of requiring a generic circuit gate and the proof π_1.

5. Produce reproducible gas measurements for the optimistic path and the dispute path, including a clear distinction between one-time deployment costs and marginal per-exchange costs.

6. Test large-file execution, especially 1 GiB inputs, using the native Rust command line pipeline when the browser/WebAssembly execution path becomes impractical.

These goals are deliberately implementation-oriented. The project does not claim a new cryptographic proof of the SOX protocol. Instead, it asks whether the theoretical variants that are attractive on paper can be translated into contracts that are measurably cheaper and still faithful to the intended protocol flow.

1.4 Methodology

The work followed an iterative methodology. The first step was to make the inherited system run reliably in a local development environment. This required rebuilding the Rust and WebAssembly components, deploying the Solidity stack, restoring the expected local database and backend folders, and understanding the actual path used by the frontend. This phase was necessary because several important costs are not visible by reading a contract alone: the application decides when a contract is deployed, which role signs which operation, whether a transaction goes through ERC-4337 tooling, and which off-chain artifacts are stored or transferred.

The second step was protocol mapping. The implementation uses function names and UI actions that do not directly match the step names in the paper. For example, the dispute trigger, challenge response, opinion submission, commitment submission, and finalization functions correspond to different substeps of the paper protocol. Before measuring gas, these mappings had to be fixed explicitly. Otherwise, two measurements with the same informal label could include different pieces of the protocol.

The third step was variant isolation. The initial engineering temptation was to add feature flags to the existing contracts. This approach is convenient for development because one contract can simulate many cases. It is not, however, an adequate gas optimization strategy: Ethereum users pay for bytecode size, storage layout, and runtime branches even when a given exchange uses only one mode. The project therefore moved progressively from a monolithic implementation to specialized contracts, clone-based optimistic escrows, and dedicated dispute games.

The fourth step was measurement. Gas was measured through Hardhat tests designed to execute complete scenarios rather than isolated functions whenever possible. This matters because deployment and initialization can dominate the total cost. For large files, off-chain timings were measured with the native Rust command line tools instead of the browser path, because the browser/WebAssembly execution path is not the right environment for 1 GiB experiments. The measurements in the final report distinguish:

- one-time infrastructure deployment costs, such as deploying factories and implementations;
- marginal optimistic costs per exchange, including deployment or clone creation and execution until completion;
- dispute initialization costs, especially the cost of deploying or creating the dispute contract;
- post-trigger dispute execution costs, such as challenge rounds and Step 8 verification;
- off-chain timings for precontract generation and dispute witness computation.

The final step was consistency checking. Several early measurements were technically correct but difficult to interpret because they mixed scopes: some came from the monolithic version, some from the specialized version, some included deployment, and some did not. The report therefore treats measurement clarity as part of the contribution. A gas table is only useful if the reader can reconstruct exactly what was paid, by whom, and at which stage of the protocol.

1.5 Main contributions

The main contributions of this project are the following.

Variant-aware optimistic contracts. The optimistic phase was split into dedicated modes for the normal case, no S-deposit, $S = B$, and $S = V$. In the final specialized implementation, the mode is selected before deployment by the frontend/backend pipeline, so the contract does not need to carry all branches at runtime. This is the main reason why the marginal optimistic cost can be reduced substantially compared with the initial account-per-exchange design.

Self-sponsored dispute execution. The dispute contracts were specialized for cases where the dispute sponsors coincide with the protocol participants, in particular $S_B = B$ and $S_V = V$. These cases matter because they remove unnecessary repetitions of the Step 7–8 loop caused by Step 9 sponsor rotation. The project therefore separates the cost of one iteration from the cost of two or three iterations, rather than hiding this effect behind a single aggregate dispute number.

Hardcoded SHA256 dispute mode. For the important case desc = SHA256, the circuit is determined by the input length. A dedicated hardcoded dispute contract was implemented so that Step 8 no longer requires the generic circuit proof π_1. In the strict specialized mode, the contract reconstructs the relevant SHA-256 gate internally; the vendor does not provide the generic gate bytes used by the monolithic verifier. This distinction is important because the monolithic hardcoded variant can remove π_1 but still requires a 64-byte gate encoding, whereas the specialized hardcoded variant removes both.

Large-file execution path. The project validated an execution path for large files up to 1 GiB using native command line tooling. This does not mean that every browser action is suitable for 1 GiB inputs. Rather, it establishes a practical split: the UI remains useful for demonstration and small end-to-end exchanges, while the native Rust pipeline is the correct measurement path for large-file precontract generation and dispute witness computation.

Reproducible gas accounting. The project produced benchmark tests for the initial, monolithic, and specialized versions. It also clarified which costs are one-time protocol deployment costs and which costs are paid per exchange. This distinction is central to evaluating whether a change is a true gas optimization or only a relocation of cost from one part of the system to another.

1.6 Scope and limitations of the project

The project focuses on implementation, measurement, and architecture. It does not attempt to replace the security proof of the SOX protocol. It also does not claim that the current code is production-ready. The implementation remains a research prototype, and several limitations remain important.

First, the current application still uses a laboratory identity model with local accounts and development-chain deployment scripts. Wallet integration would be necessary for a realistic user-facing deployment, but it was not the main objective of this project.

Second, large-file execution is validated through the native command line pipeline, not through a fully browser-based flow. This is intentional: the original browser/WebAssembly path is convenient for small demonstrations, but it is not the right tool for measuring 1 GiB precontract and dispute computations.

Third, the project revealed questions about the inherited circuit encoding and Merkle tree domain separation. These questions concern the historical circuit format and are not caused by the gas-specialization work itself, but they are relevant for any future production-grade version. A clean final design should make gate encodings unambiguous and should separate Merkle leaves from internal nodes by domain tags.

Finally, some architectural improvements remain future work. In particular, a more radical factory or registry architecture could avoid deploying a full optimistic account for every exchange. The present project already moves in that direction with specialized contracts and clones, but a fully reusable registry-style deployment model would require a broader redesign of the inherited application.

1.7 Report structure

This first chapter defines the problem, the terminology, the goals, and the methodology of the project. The remaining chapters will describe the inherited implementation, the protocol variants, the specialized contract architecture, the experimental measurements, the security and encoding issues identified during the project, and the installation and demonstration workflow. The rest of the report will keep the terminology introduced in Table 1.1 so that gas costs can be compared without mixing the initial, monolithic, and specialized implementations.

Bibliography

[1] Anonymous Author(s). *Sponsored Fair Exchange*. Manuscript provided as project reference, 2026.

[2] Hana El Moutaoukil. *Sponsored Fair Exchange: Implementation*. Semester project report, EPFL/LASEC, September 2025.